ABSTRACT FACTORY

Samuel García
Samuel Riaño

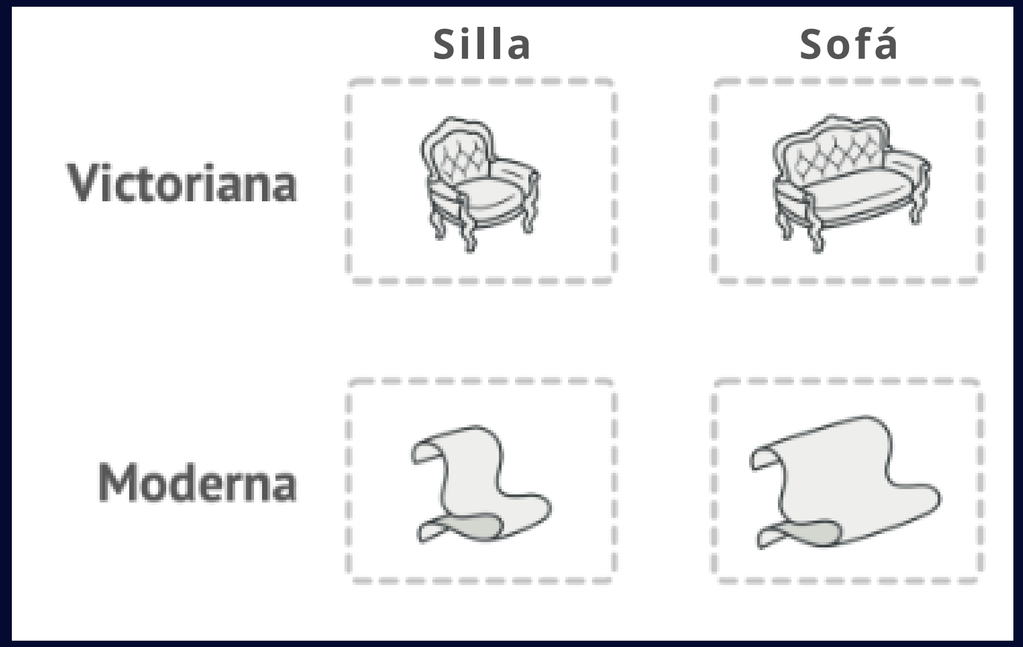
Docente: Juan Ricardo Cogollo
Ing. de sistemas, POO

¿QUE ES?

El Abstract Factory es un patrón de diseño creacional que proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.

En lugar de instanciar objetos directamente con new, el cliente utiliza una fábrica abstracta, la cual delega la creación a fábricas concretas, cada una encargada de producir una familia específica de objetos.

EJEMPLO



Imagina que estás construyendo una fábrica de muebles. No quieres solo una silla, quieres una familia de productos que combinen entre sí (por ejemplo: una silla y un sofá, todos en estilo "Moderno" o todos en estilo "Victoriano").

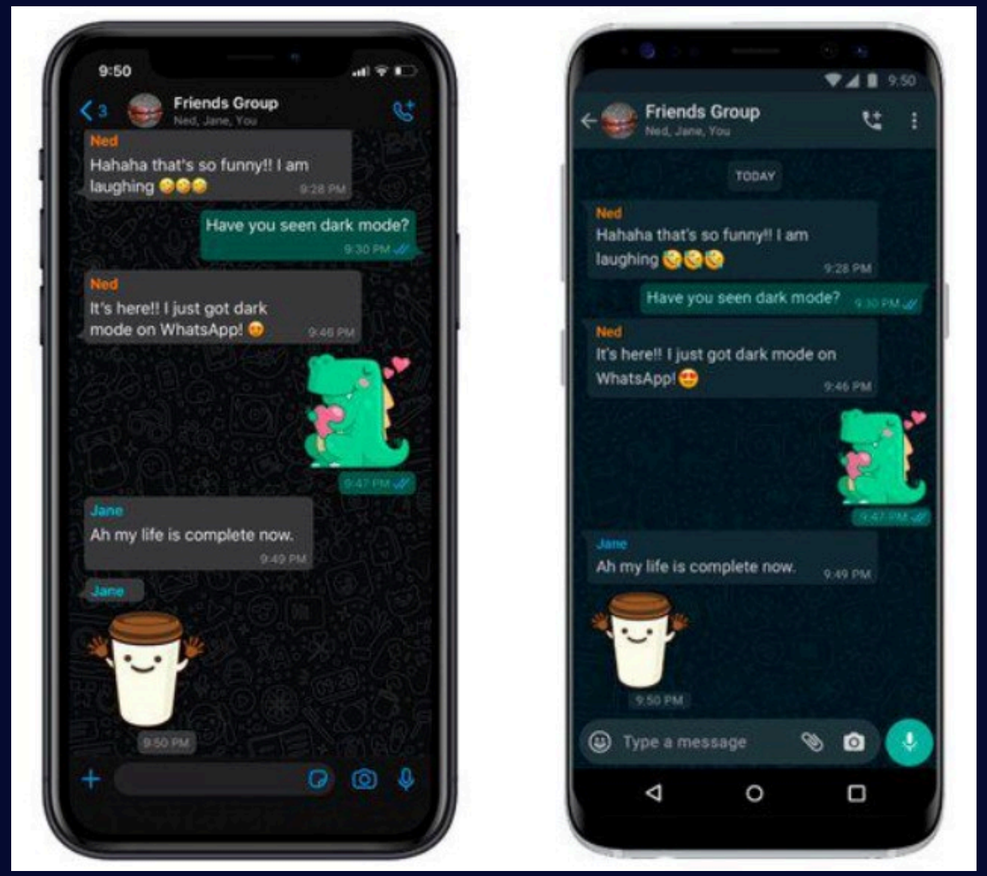
Necesitamos una forma de crear objetos individuales de mobiliario para que combinen con otros objetos de la misma familia. Además, no queremos cambiar el código existente al añadir al programa nuevos productos.

```cpp
// --- Productos ---
class SillaModerna : public Silla { /* ... */ };
class SillaVictoriana : public Silla { /* ... */ };
class SofaModerno : public Sofa { /* ... */ };
class SofaVictoriano : public Sofa { /* ... */ };

// --- Fábricas Individuales (El desorden) ---
class FabricaSillaModerna {
public:
    Silla* crear() { return new SillaModerna(); }
};

class FabricaSillaVictoriana {
public:
    Silla* crear() { return new SillaVictoriana(); }
};

class FabricaSofaModerno {
public:
    Sofa* crear() { return new SofaModerno(); }
};

class FabricaSofaVictoriano {
public:
    Sofa* crear() { return new SofaVictoriano(); }
};
```

```cpp
// --- INTERFAZ DE LA FÁBRICA ---
class FabricaMuebles {
public:
    // Métodos virtuales puros = Interfaz
    virtual Silla* crearSilla() = 0;
    virtual Sofa* crearSofa() = 0;
};

// --- Fábricas Concretas (Agrupan por estilo) ---
class FabricaModerna : public FabricaMuebles {
public:
    Silla* crearSilla() override { return new SillaModerna(); }
    Sofa* crearSofa() override { return new SofaModerno(); }
};

class FabricaVictoriana : public FabricaMuebles {
public:
    Silla* crearSilla() override { return new SillaVictoriana(); }
    Sofa* crearSofa() override { return new SofaVictoriana(); }
};
```

OTRO EJEMPLO

Imagina que estás programando una app que debe funcionar en ambos sistemas: android y ios. No quieres escribir código diferente para cada botón ni elemnto en la app; quieres que el sistema detecte dónde está y cree automáticamente los componentes correctos.

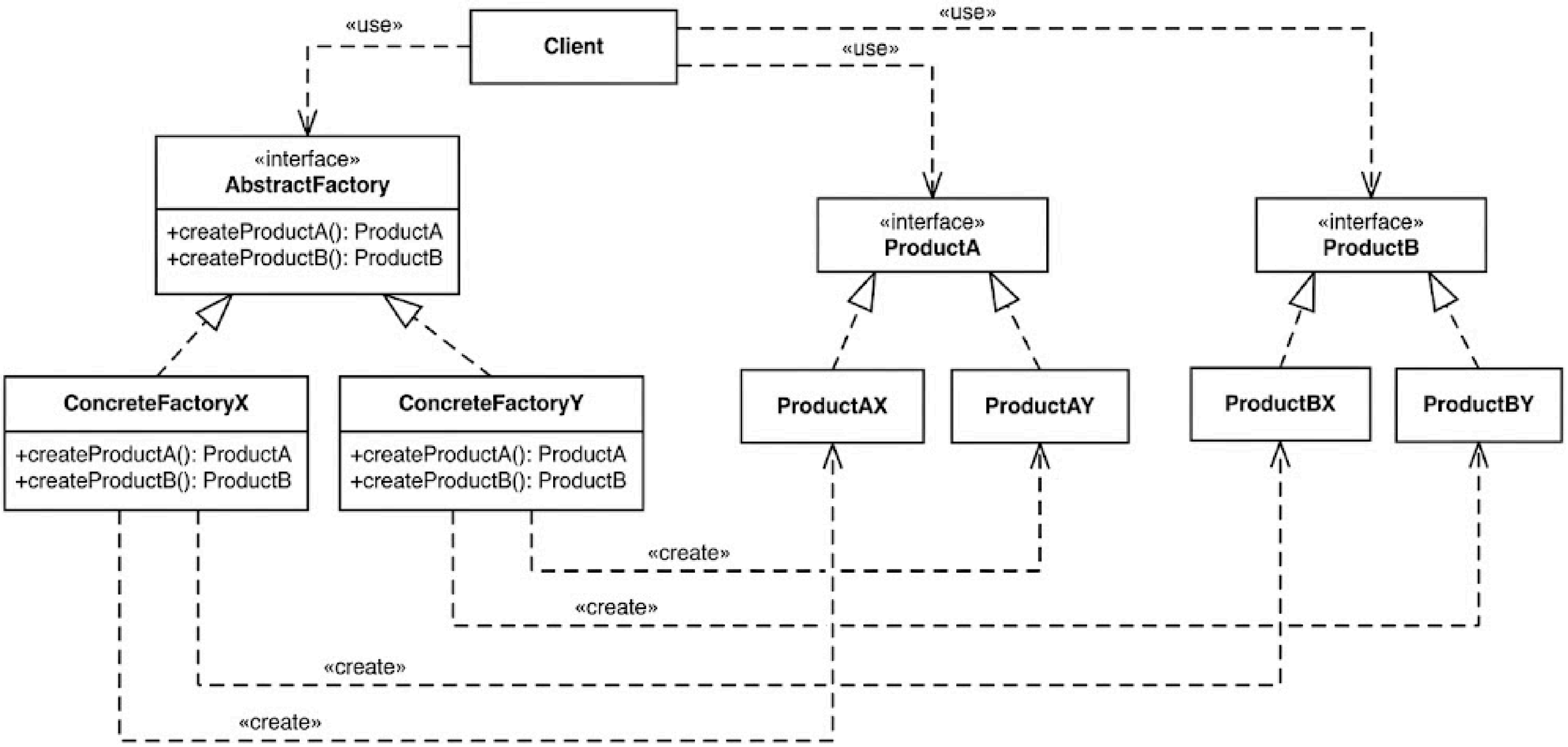


Necesitamos crear una interfaz de factoría abstracta que maneje los dos sistemas ios y android y que puedan crear cada uno sus propios elementos, botones, etc...

```cpp
// --- Productos ---
class BotonAndroid : public Boton { /* ... */ };
class BotonIOS : public Boton { /* ... */ };
class MenuAndroid : public Menu { /* ... */ };
class MenuIOS : public Menu { /* ... */ };

// --- El desorden de fábricas ---
class FabricaBotonAndroid {
public:
    Boton* crear() { return new BotonAndroid(); }
};

class FabricaBotonIOS {
public:
    Boton* crear() { return new BotonIOS(); }
};

class FabricaMenuAndroid {
public:
    Menu* crear() { return new MenuAndroid(); }
};
```

```cpp
// --- INTERFAZ DE LA FÁBRICA (El contrato) ---
class FabricaUI {
public:
    virtual Boton* crearBoton() = 0;
    virtual Menu* crearMenu() = 0;
};

// --- Fábricas Concretas por Sistema ---
class FabricaAndroid : public FabricaUI {
public:
    Boton* crearBoton() override { return new BotonAndroid(); }
    Menu* crearMenu() override { return new MenuAndroid(); }
};

class FabricaIOS : public FabricaUI {
public:
    Boton* crearBoton() override { return new BotonIOS(); }
    Menu* crearMenu() override { return new MenuIOS(); }
};
```

DIAGRAMA DE CLASE



GRACIAS